Systolic Array Scheduling for Sparse Matrix Workloads on Wafer-Scale Chips

A. Testwood, J. Kimani, and R. Okafor — Department of Computer Architecture, Institute of Parallel Systems

Abstract — Sparse matrix workloads dominate inference serving for large language models, yet commodity accelerators schedule them with dense-array primitives that waste up to 71% of available multiply-accumulate throughput. We present a scheduling discipline for systolic arrays that exploits block-level sparsity metadata computed once at graph compilation. Across 14 transformer workloads, our scheduler improves effective throughput by 2.3x at iso-accuracy and reduces time-to-first-token by 38% relative to dense padding baselines. We further characterize the interaction between sparsity-aware scheduling and memory bandwidth, showing that sub-8% dense blocks should remain padded under current HBM3 bandwidth ceilings.

1. Introduction

Wafer-scale inference chips promise an order-of-magnitude reduction in served-token cost, but only if their compute fabric can be kept busy. Sparse activations break that assumption: after structured pruning, transformer blocks exhibit 40–85% zeros arranged in irregular patterns that defeat both dense systolic schedules and naive column-skipping heuristics. Prior work either compiles sparsity into static masks that lose efficiency as distributions drift, or defers to runtime compaction whose coordination overhead negates its gains below batch 32.

Our approach is deliberately conservative: the hardware remains a standard systolic array, and all scheduling decisions are made by an offline pass that emits a compressed schedule table consumed by the array controller. This separates the correctness argument (schedule validity) from the performance argument (metadata quality), letting us certify safety with a 200-line checker while the scheduler itself iterates freely.

2. Related Work

Sparsity-aware accelerators fall into three families. Fixed-mask designs such as sparse tensor cores accept whole-block zeros but cannot adapt to dynamic distributions. Software pipelining approaches reorder matrix blocks for locality without exploiting zeros. Runtime-compaction systems skip zeros dynamically at the cost of control divergence. Our scheduler is closest to the third family but moves all decisions to compile time, eliminating divergence while retaining adaptability through schedule regeneration.

3. Schedule Construction

Given a block sparsity profile B (per 8x8 tile), the scheduler solves a bin-packing variant over systolic wavefronts: tiles are assigned to wavefront slots so that (a) no two tiles sharing an operand row contend for the same broadcast bus, and (b) high-density tiles are scheduled in the first $W/2$ wavefronts to maximize early occupancy. We relax the NP-hard assignment with a two-phase greedy that we show is within 12% of an ILP optimum on all profiles with density variance below 0.3.

4. Evaluation

We evaluate on a 7x14 systolic tile simulator validated against published silicon measurements (mean error 3.1%). On LLaMA-class decoder blocks at batch 8, the scheduler sustains 2.3x effective MAC utilization versus dense padding and 1.6x versus runtime compaction. Time-to-first-token improves by 38% because prefill phases, dominated by wide sparse matmuls, complete in fewer wavefronts. Bandwidth sensitivity analysis shows the gains compress to 1.2x when HBM bandwidth is halved, indicating scheduling value is compute-bound, not bandwidth-bound.

5. Limitations

The schedule table must be regenerated when sparsity distributions drift beyond a KL-divergence of 0.15 from the compiled profile; regeneration takes 40ms per layer on the host CPU. Extremely fine-grained (per-element) sparsity remains out of scope: below tile density 0.08, the metadata overhead exceeds the compute savings and padding remains preferable.

6. Conclusion

Compile-time scheduling is a practical middle path for sparse inference on systolic arrays: it captures most of runtime compaction's gains without its control overhead, and it composes with existing dense execution as a fallback. We release our scheduler, checker, and all 14 workload traces to support replication.